TALLER · 16 NOV 2026 · 5 H

IA aplicada a un entorno físico

Enseñamos a andar a un robot, sin tocar un solo tornillo.



Guía práctica · robot Microduck · simulación en MuJoCo

LO PRIMERO DE TODO · 8 MIN

Prepara tu PC ahora

1

Carpeta

Crea una carpeta vacía y ábrela en VS Code.

2

Clonar

En la terminal de VS Code, copia el proyecto con git clone.

3

Instalar

Doble clic en INSTALAR.bat. Unos 3 minutos, sin internet.

4

Seguir

Sigue sola mientras empieza el taller.

git clone https://github.com/david2526IA/workshop-ia-fisica .

Sabrás que ha terminado cuando la ventana ponga TODO LISTO.

Las cinco horas

Hora	Bloque	Tú haces
0:00	Ignición	Mirar y contestar
0:20	1 · El cerebro	Playground y el cerebro del pato
1:15	2 · Del plano al gemelo digital	Cambiar el suelo
1:45	3 · Tu laboratorio	Comprobar tu instalación
2:45	Descanso	Café
2:55	4 · Laboratorio	Perturbar el robot y reto
4:25	5 · De la simulación a la realidad	Pensar qué falla fuera
4:45	Cierre	Resumen

Cuatro carpetas, un laboratorio

robot/

El cuerpo: mallas, motores y mundo.

policies/

El cerebro: ficheros .onnx.

exercises/

Tus ejercicios.

scripts/

El programa que une cuerpo y cerebro.

Además: .venv es una caja aislada de Python, requirements.txt es la lista de la compra y Git guarda la historia.

TU TURNO · 30 MIN

Comprueba tu laboratorio

python check_setup.py --viewer

Debe acabar en **TODO LISTO** y abrir el robot 5 segundos.

Después lanza **run_demo.bat** y mira al robot andar con IA.

Contraste: **n2_02_sin_cerebro.py** cae en menos de un segundo.



Cinco etapas, de un plano a algo que se mueve

1

Modelo 3D

Las piezas en mallas .stl.

2

URDF

Árbol de piezas unidas por articulaciones.

3

MJCF

Masas, colisiones y motores.

4

Escena

Suelo, luz y posturas guardadas.

5

Validación

Se tiene en pie y se mueve.

En el repo: robot/assets, robot_groundcontact.xml y scene.xml. La conversión desde Onshape se explica, no se ejecuta.

TU TURNO

Cambia el mundo del robot

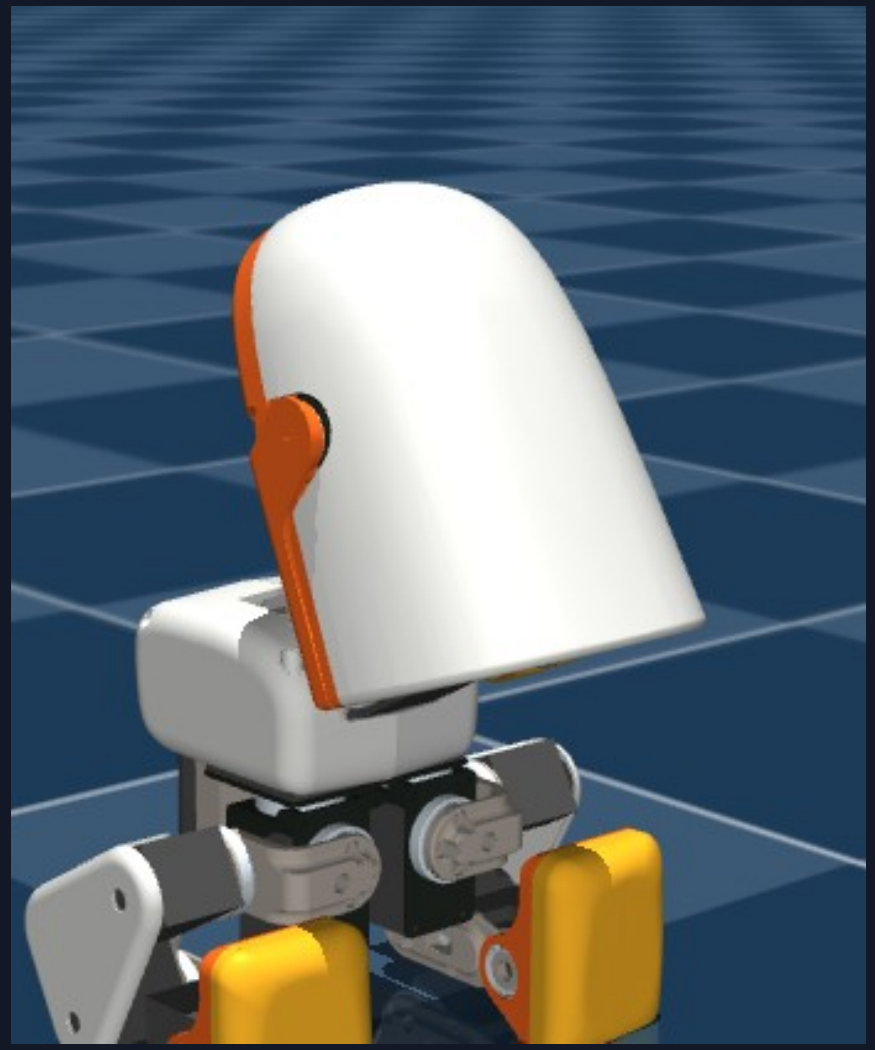
1. Abre robot/scene.xml en VS Code.

2. Busca groundplane y cambia rgb1 y rgb2: los colores del suelo.

3. Guarda con Ctrl + S.

4. Lanza el visor y mira el resultado.

python exercises/n1_00_ver_robot.py

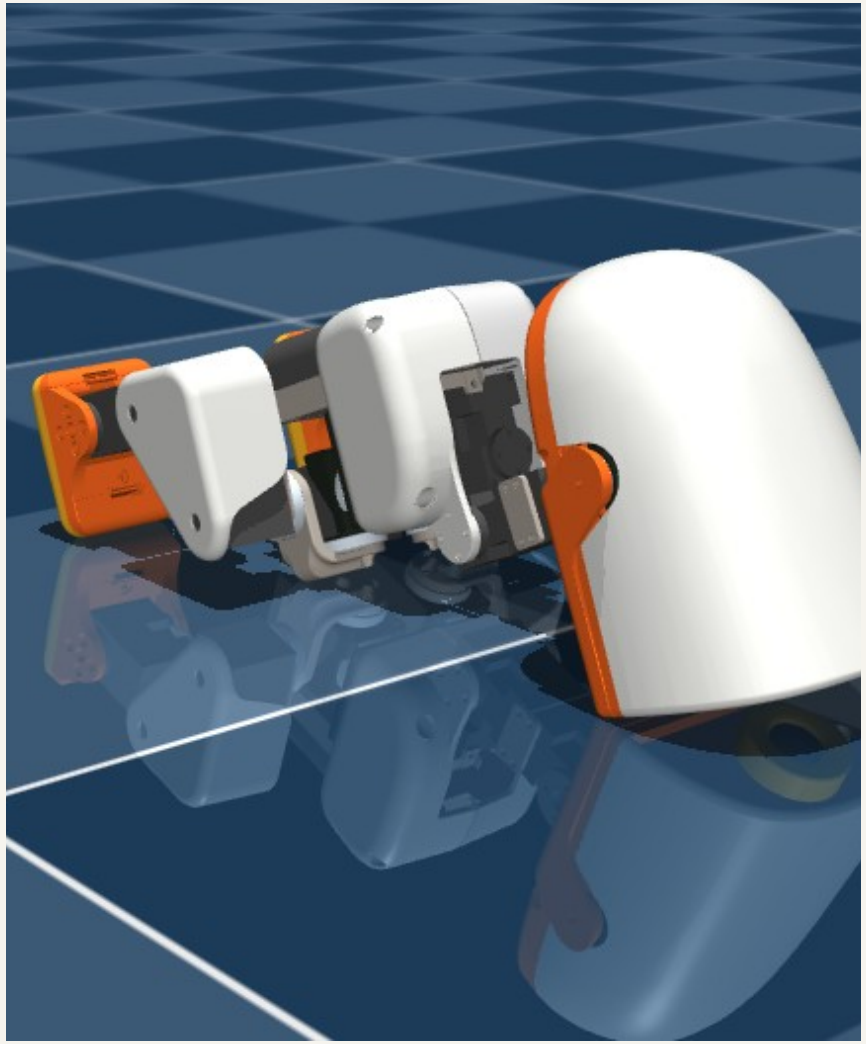


Aprender es como montar en bici

Nadie te da ecuaciones: te caes, ajustas y repites.

1 · Intentar 2 · Ver el error 3 · Ajustar

Toda la IA del taller es este bucle.



Una red neuronal es una función con botones

Entra

Lo que siente el robot: 61 números.

➡

La red

Miles de botones: los pesos.

➡

Sale

Lo que hacen los motores: 14 ángulos.

Aprender es ajustar los botones hasta que el resultado sea bueno.

Error y ajuste

El error mide lo mal que va la red.

El gradiente es la brújula: dice hacia dónde mover cada botón.

Cada paso mueve los botones un poco en la dirección que baja el error.

Pruébalo: en n2_00_cerebro_en_vivo.py quítale un sentido al pato y mira qué pasa.

Aprendizaje por refuerzo: cuatro piezas

Agente

Quien decide: la política del robot.

Entorno

El mundo donde actúa: el simulador.

Acción

14 ángulos de motor, 50 veces por segundo.

Recompensa

Un número: seguir la velocidad pedida y no caerse.

No hay profesor con la respuesta correcta, solo recompensas.

Recompensa frente a imitación

Imitar a un humano

Es rápido, pero nunca lo supera.

Aprender con recompensa

Puede superarlo, pero diseñar la recompensa es difícil.

Si solo premias «avanzar», puede aprender a arrastrarse o a lanzarse hacia delante.

TU TURNO · 25 MIN

Toca el cerebro

1. Abre VS Code y la carpeta del taller.

2. Entra en playground.tensorflow.org y añade neuronas.

3. Ejecuta n2_00: mira dentro del cerebro del pato y quítale un sentido.

4. Ejecuta n2_01: mira la nota que recibe y cambia los pesos.

¿Qué sentido le importa más para no caerse?



El mismo robot, distintos cerebros

NIVEL 0	NIVEL 1	NIVEL 2	NIVEL 3	NIVEL 4	NIVEL 5
Recién nacido	**Quieto**	**Tambaleante**	**Bien**	**Mejor**	**Oficial**
Se cae en un segundo.	Se mantiene, pero no avanza.	Avanza, pero se desvía.	Avanza con soltura.	Anda lejos y sin caerse.	El cerebro final de Pollen.

Cambia el cerebro con el nivel de n3_01_progresion.py.

Perturba el mundo, de uno en uno

Opción	Qué simula	Prueba
friction en scene.xml	El suelo resbala	1, 0.3, 0.1
--vin	Batería baja	6.5
--delay	Motores con retraso	2 4
--action-scale	Motores débiles	0.7
--walking	Otro cerebro	progresion_*.onnx

Empuja con Ctrl + clic derecho. ¿Cuándo empieza a fallar?

BLOQUE 4 · LABORATORIO

Tu cuaderno de misiones

A · 10 MIN

Ver actuar

Anda, gira, siéntate y haz la voltereta.

B · 20 MIN

Perturbar

Empuja, hielo, batería baja, motores débiles.

C · 10 MIN

Comparar

Sin cerebro frente a con cerebro, y los seis niveles.

D · 25 MIN

Entrenar

Predice y compara curvas reales.

EXTRA

¡A patinar!

El mismo robot con ruedas y otro cerebro.

Anota lo que ves en MISIONES.md. Lo importante es qué cambia y por qué.

La configuración de un entrenamiento

```
# tarea
Mjlab-StandUp-Flat-MicroDuck
# entorno  (env.yaml)
num_envs: 4096          # robots a la vez
decimation: 4 × 0,005 s  # = 50 Hz
episode_length_s: 6
# algoritmo PPO  (agent.yaml)
learning_rate: 0.001  (adaptive)
gamma: 0.99    lam: 0.95
clip_param: 0.2    entropy_coef: 0.01
num_learning_epochs: 5    num_mini_batches: 4
num_steps_per_env: 24
# red  (actor y critic)
hidden_dims: [512, 256, 128]   activation: elu
```

Dos ficheros, todo el entrenamiento

Cada carrera guarda su configuración en params/env.yaml y params/agent.yaml.

Cualquier valor se cambia al lanzar:

--agent.algorithm.learning-rate 0.0005

--env.scene.num-envs 2048

--agent.max-iterations 500

Cada opción del laboratorio es un problema real

--vin

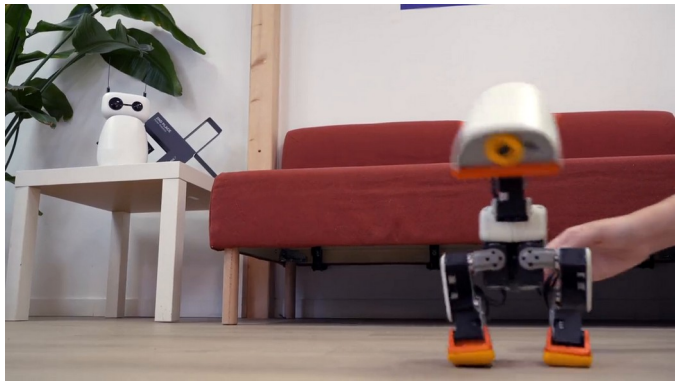
La batería cae de voltaje.

--delay

La comunicación llega tarde.

friction

El suelo real no es el del simulador.



El robot real se recupera de un empujón

Lo difícil no es entrenar en el simulador, sino que funcione fuera.

Vídeo: Pollen Robotics (press kit oficial).

Tres pasos para ponerlo a entrenar

1

Preparar

Instala todo y comprueba que la GPU funciona.

2

Entrenar

4096 robots aprenden a la vez, unos 10 minutos.

3

Mirar

Si la recompensa sube, el robot está aprendiendo.

Scripts de la carpeta entrenar/ (ver ENTRENAR.md). Hace falta una GPU NVIDIA.

Lo que te llevas

Aprender = error y ajuste.

Refuerzo = recompensa.

Gemelo digital = cuerpo virtual.

Política = cerebro.

Sim-to-real = el reto.

Recursos: tu repo, pollen-robotics/microduck_rl, Hugging Face (pollen-robotics/microduck-policies) y la documentación de MuJoCo.

